Start coding or generate with AI.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

"""
Project Pipeline Library Overview:
- torch & torch.nn: Core framework for building and training the CNN architect
- torch.optim: Provides optimization algorithms like Adam to update network we
- torch.utils.data: Essential for creating DataLoaders to manage batches durir
- pandas: Used for data ingestion from Excel and initial data manipulation.
- numpy: Facilitates numerical operations and efficient handling of the GloVe
- sklearn.model_selection: Used for partitioning the dataset into training and
- sklearn.metrics: Provides tools for quantitative model evaluation (accuracy,
- matplotlib & seaborn: Used for visualizing training history and confusion ma
"""

print('Libraries imported successfully.')
```

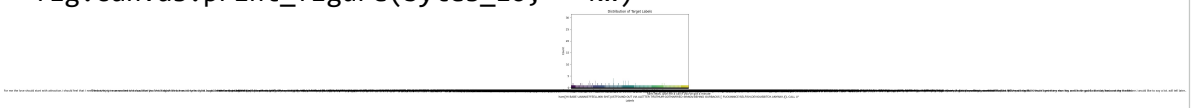
```
Libraries imported successfully.
```

```
df = pd.read_excel('/content/Book1.xlsx')

print('--- Initial Data Overview ---')
print(df.head())
print('\n--- Data Types and Missing Values ---')
print(df.info())

text_col = df.columns[0]
label_col = df.columns[1]
print(f'\nDetected Text Column: {text_col}')
print(f'Detected Label Column: {label_col}')

df = df.dropna(subset=[text_col, label_col])

print('\n--- Label Distribution ---')
label_counts = df[label_col].value_counts()
print(label_counts)

num_classes = df[label_col].nunique()
```

```
classification_type = 'Binary' if num_classes == 2 else 'Multiclass'
print(f'\nClassification Type: {classification_type} ({num_classes} classes)

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x=label_col, palette='viridis')
plt.title('Distribution of Target Labels')
plt.xlabel('Labels')
plt.ylabel('Count')
plt.show()
```

```
--- Initial Data Overview ---
    ham  \
0   ham
1  spam
2   ham
3   ham
4  spam

  Go until jurong point, crazy.. Available only in bugis n great world la e b
0                      Ok lar... Joking wif u oni...
1  Free entry in 2 a wkly comp to win FA Cup fina...
2  U dun say so early hor... U c already then say...
3  Nah I don't think he goes to usf, he lives aro...
4  FreeMsg Hey there darling it's been 3 week's n...

--- Data Types and Missing Values ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5571 entries, 0 to 5570
Data columns (total 2 columns):
 #   Column
---  ------
 0   ham
 1   Go until jurong point, crazy.. Available only in bugis n great world la
dtypes: object(2)
memory usage: 87.2+ KB
None

Detected Text Column: ham
Detected Label Column: Go until jurong point, crazy.. Available only in bugis

--- Label Distribution ---
Go until jurong point, crazy.. Available only in bugis n great world la e buf
Sorry, I'll call later
I cant pick the phone right now. Pls send a message
Ok...
Wen ur lovable bcums angry wid u, dnt take it seriously.. Coz being angry is
Ok.

Dear 0776xxxxxxx U've been invited to XCHAT. This is our final attempt to con
I think asking for a gym is the excuse for lazy people. I jog.
I love your ass! Do you enjoy doggy style? :)
Come to me, slave. Your doing it again ... Going into your shell and unconsci
CALL 09090900040 & LISTEN TO EXTREME DIRTY LIVE CHAT GOING ON IN THE OFFICE R
Name: count, Length: 5168, dtype: int64
```

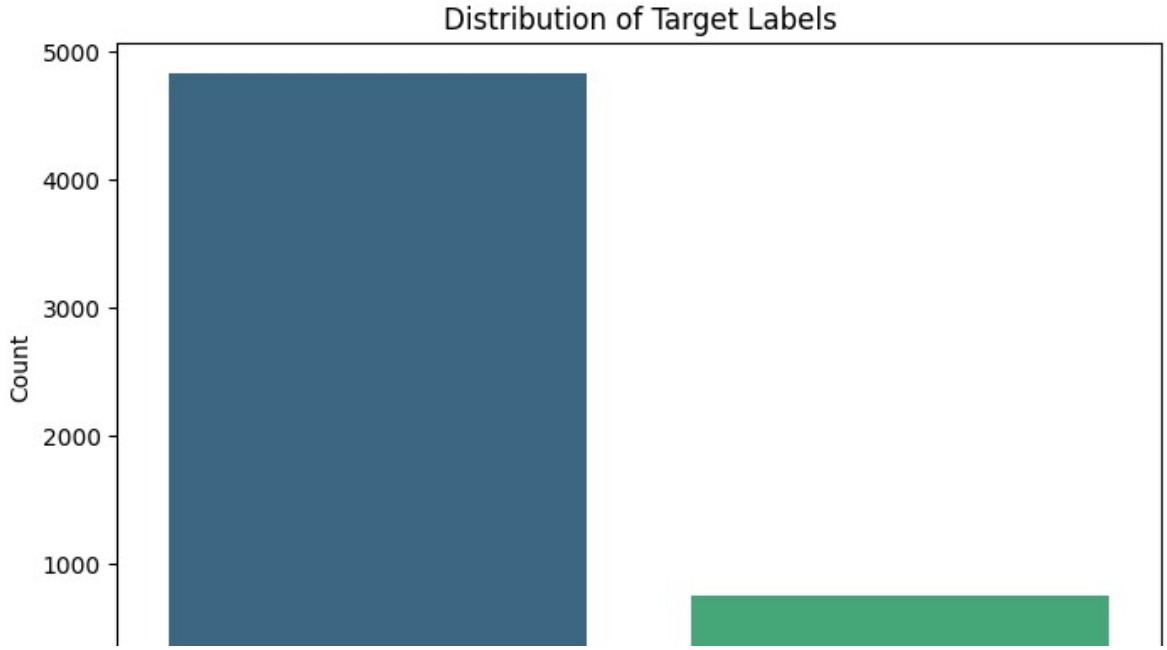
```
Name: count, Length: 5168, dtype: int64

Classification Type: Multiclass (5168 classes)
/tmp/ipython-input-1354461334.py:24: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i

  sns.countplot(data=df, x=label_col, palette='viridis')
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserW
  fig.canvas.print_figure(bytes_io, **kw)
```




```
df = pd.read_excel('/content/Book1.xlsx', header=None)

# Assign logical names based on inspection: Column 0 is Label, Column 1 is T
df.columns = ['label', 'text']

print('--- Initial Data Overview ---')
print(df.head())
print('\n--- Data Types and Missing Values ---')
print(df.info())

text_col = 'text'
label_col = 'label'

df = df.dropna(subset=[text_col, label_col])

print('\n--- Label Distribution ---')
label_counts = df[label_col].value_counts()
print(label_counts)

num_classes = df[label_col].nunique()
classification_type = 'Binary' if num_classes == 2 else 'Multiclass'
print(f'\nClassification Type: {classification_type} ({num_classes} classes)
```

```
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x=label_col, hue=label_col, palette='viridis', legend
plt.title('Distribution of Target Labels')
plt.xlabel('Labels')
plt.ylabel('Count')
plt.show()
```

```
--- Initial Data Overview ---
  label                                               text
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

--- Data Types and Missing Values ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   label   5572 non-null   object
 1   text    5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
None

--- Label Distribution ---
label
ham     4825
spam     747
Name: count, dtype: int64

Classification Type: Binary (2 classes)
```



```
import string

# 1. Define cleaning function
def clean_text(text):
    text = str(text).lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text

# 2. Apply cleaning function
df['clean_text'] = df['text'].apply(clean_text)

# 3. Convert labels to numerical values
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

# 4. Build vocabulary
all_tokens = [word for text in df['clean_text'] for word in text.split()]
vocab = {word: i + 1 for i, word in enumerate(sorted(set(all_tokens)))}
vocab_size = len(vocab) + 1

# 5. Transform text to sequences
def text_to_sequence(text):
    return [vocab[word] for word in text.split() if word in vocab]

df['sequences'] = df['clean_text'].apply(text_to_sequence)

# 6. Determine max length and pad sequences
lengths = df['sequences'].apply(len)
max_len = int(np.percentile(lengths, 95))

print(f'Vocabulary Size: {vocab_size}')
print(f'Max Sequence Length (95th percentile): {max_len}')

def pad_sequence(seq, max_length):
    if len(seq) > max_length:
        return np.array(seq[:max_length])
    return np.pad(seq, (0, max_length - len(seq)), 'constant')

X = np.stack(df['sequences'].apply(lambda x: pad_sequence(x, max_len)).value
y = df['label_num'].values

print(f'Input shape (X): {X.shape}')
```

```
print(f'Target shape (y): {y.shape}')
```

```
Vocabulary Size: 9662
Max Sequence Length (95th percentile): 32
Input shape (X): (5572, 32)
Target shape (y): (5572,)
```

```
embedding_dim = 300
glove_path = '/content/glove.6B.300d.txt'
embeddings_index = {}

print(f'Loading GloVe embeddings from {glove_path}...')
with open(glove_path, 'r', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs

print(f'Found {len(embeddings_index)} word vectors.')

# Create the embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in vocab.items():
    if i < vocab_size:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector

print(f'Embedding Matrix Shape: {embedding_matrix.shape}')
if embedding_matrix.shape[0] == vocab_size:
    print('Verification Successful: Matrix rows match vocab_size.')
else:
    print('Verification Failed: Matrix row mismatch.')
```

```
Loading GloVe embeddings from /content/glove.6B.300d.txt...
Found 44125 word vectors.
Embedding Matrix Shape: (9662, 300)
Verification Successful: Matrix rows match vocab_size.
```

```
# 1. Split the data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

# 2. Convert to PyTorch tensors
# Features are Long (indices) and labels are Long (for CrossEntropyLoss)
X_train_t = torch.tensor(X_train, dtype=torch.long)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t = torch.tensor(X_test, dtype=torch.long)
y_test_t = torch.tensor(y_test, dtype=torch.long)

# 3. Create TensorDatasets
```

```
# 3. Create TensorDatasets
train_dataset = TensorDataset(X_train_t, y_train_t)
test_dataset = TensorDataset(X_test_t, y_test_t)

# 4. Initialize DataLoaders
batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f'Training samples: {len(train_dataset)}')
print(f'Testing samples: {len(test_dataset)}')
print(f'Batch size: {batch_size}')
print('DataLoaders prepared successfully.')
```

```
Training samples: 4457
Testing samples: 1115
Batch size: 32
DataLoaders prepared successfully.
```

Double-click (or enter) to edit

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, embedding_matrix, num_clas
        super(TextCNN, self).__init__()
        # 1. Embedding layer initialized with GloVe
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.embedding.weight = nn.Parameter(torch.tensor(embedding_matrix,
        self.embedding.weight.requires_grad = False  # Freeze embeddings

        # 2. Convolutional layers with different kernel sizes
        self.conv1 = nn.Conv1d(in_channels=embedding_dim, out_channels=128,
        self.conv2 = nn.Conv1d(in_channels=embedding_dim, out_channels=128,

        # 3. Pooling and Dropout
        self.pool = nn.AdaptiveMaxPool1d(1)
        self.dropout = nn.Dropout(0.5)

        # 4. Fully connected layer (combining outputs from both conv layers)
        self.fc = nn.Linear(128 * 2, num_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        # x shape: [batch_size, seq_len]
        x = self.embedding(x)  # [batch_size, seq_len, embedding_dim]

        # Permute for Conv1d: [batch_size, channels, length]
        x = x.permute(0, 2, 1)

        # Apply convolutions and pooling
        x1 = self.relu(self.conv1(x))
```

```
        x1 = self.pool(x1).squeeze(-1) # [batch_size, 128]

        x2 = self.relu(self.conv2(x))
        x2 = self.pool(x2).squeeze(-1) # [batch_size, 128]

        # Concatenate features
        combined = torch.cat((x1, x2), dim=1)

        # Dropout and output
        out = self.dropout(combined)
        logits = self.fc(out)
        return logits

# Instantiate the model
model = TextCNN(vocab_size=vocab_size,
                embedding_dim=embedding_dim,
                embedding_matrix=embedding_matrix,
                num_classes=num_classes)

print(model)
print('\nModel architecture defined successfully.')
```

```
TextCNN(
  (embedding): Embedding(9662, 300)
  (conv1): Conv1d(300, 128, kernel_size=(3,), stride=(1,))
  (conv2): Conv1d(300, 128, kernel_size=(5,), stride=(1,))
  (pool): AdaptiveMaxPool1d(output_size=1)
  (dropout): Dropout(p=0.5, inplace=False)
  (fc): Linear(in_features=256, out_features=2, bias=True)
  (relu): ReLU()
)

Model architecture defined successfully.
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

# 1. Initialize Loss Function and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 2. Training and Validation Loop
num_epochs = 15
history = {'train_loss': [], 'val_loss': [], 'val_acc': []}

print(f'Starting training on {device}...\n')

for epoch in range(num_epochs):
    # --- Training Phase ---
    model.train()
    running_loss = 0.0
```

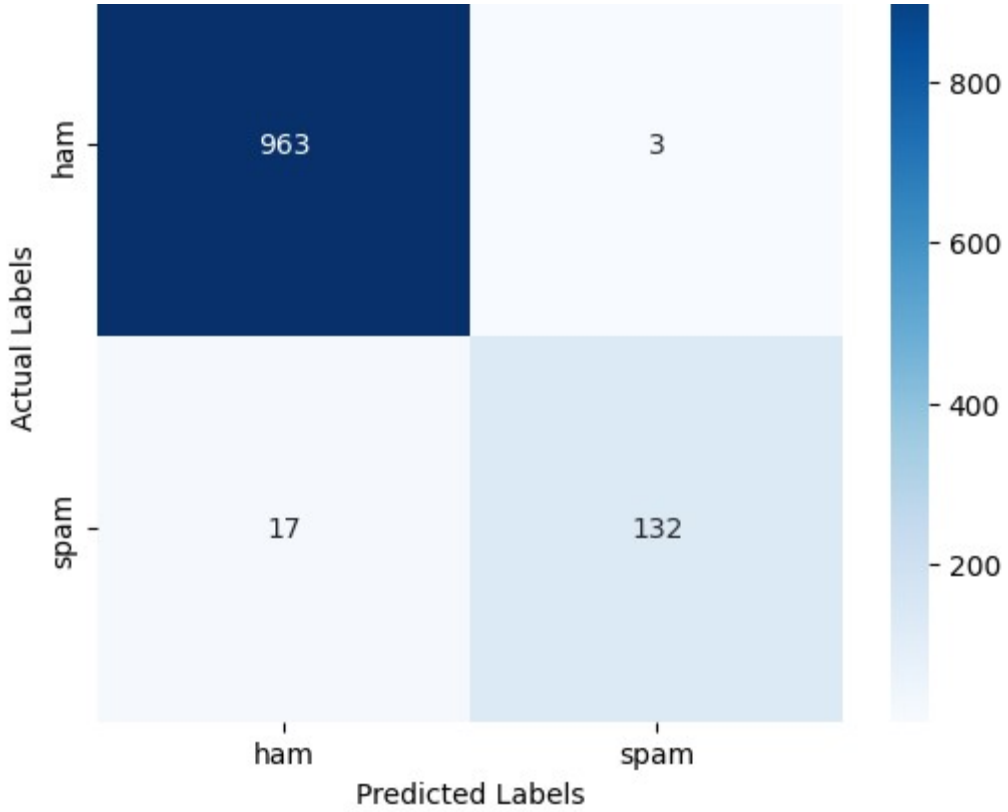
```
    for texts, labels in train_loader:
        texts, labels = texts.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(texts)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * texts.size(0)

    epoch_train_loss = running_loss / len(train_loader.dataset)

    # --- Validation Phase ---
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for texts, labels in test_loader:
            texts, labels = texts.to(device), labels.to(device)
            outputs = model(texts)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * texts.size(0)

            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    epoch_val_loss = val_loss / len(test_loader.dataset)
    epoch_val_acc = (correct / total) * 100

    # Store history
    history['train_loss'].append(epoch_train_loss)
    history['val_loss'].append(epoch_val_loss)
    history['val_acc'].append(epoch_val_acc)

    print(f'Epoch [{epoch+1}/{num_epochs}] - Train Loss: {epoch_train_loss:.

print('\nTraining complete.')
```

```
Starting training on cpu...

Epoch [1/15] - Train Loss: 0.1828 - Val Loss: 0.0924 - Val Acc: 96.41%
Epoch [2/15] - Train Loss: 0.0623 - Val Loss: 0.0718 - Val Acc: 97.67%
Epoch [3/15] - Train Loss: 0.0309 - Val Loss: 0.0590 - Val Acc: 98.03%
Epoch [4/15] - Train Loss: 0.0172 - Val Loss: 0.0582 - Val Acc: 98.30%
Epoch [5/15] - Train Loss: 0.0143 - Val Loss: 0.0587 - Val Acc: 98.39%
Epoch [6/15] - Train Loss: 0.0090 - Val Loss: 0.0547 - Val Acc: 98.39%
Epoch [7/15] - Train Loss: 0.0066 - Val Loss: 0.0724 - Val Acc: 98.21%
Epoch [8/15] - Train Loss: 0.0045 - Val Loss: 0.0683 - Val Acc: 98.48%
Epoch [9/15] - Train Loss: 0.0031 - Val Loss: 0.0604 - Val Acc: 98.30%
```

```
Epoch [9/15] - Train Loss: 0.0031 - Val Loss: 0.0664 - Val Acc: 98.30%
Epoch [10/15] - Train Loss: 0.0023 - Val Loss: 0.0837 - Val Acc: 98.30%
Epoch [11/15] - Train Loss: 0.0026 - Val Loss: 0.0790 - Val Acc: 98.21%
Epoch [12/15] - Train Loss: 0.0020 - Val Loss: 0.0711 - Val Acc: 98.21%
Epoch [13/15] - Train Loss: 0.0014 - Val Loss: 0.0627 - Val Acc: 98.48%
Epoch [14/15] - Train Loss: 0.0014 - Val Loss: 0.0802 - Val Acc: 98.39%
Epoch [15/15] - Train Loss: 0.0032 - Val Loss: 0.0948 - Val Acc: 98.21%

Training complete.
```

```
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for texts, labels in test_loader:
        texts, labels = texts.to(device), labels.to(device)
        outputs = model(texts)
        _, predicted = torch.max(outputs, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Generate Classification Report
target_names = ['ham', 'spam']
print('--- Classification Report ---')
print(classification_report(all_labels, all_preds, target_names=target_names

# Generate Confusion Matrix
cm = confusion_matrix(all_labels, all_preds)

# Visualize Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.title('Confusion Matrix: Ham vs Spam')
plt.xlabel('Predicted Labels')
plt.ylabel('Actual Labels')
plt.show()
```

```
--- Classification Report ---
              precision    recall  f1-score   support

         ham       0.98      1.00      0.99       966
        spam       0.98      0.89      0.93       149

    accuracy                           0.98      1115
   macro avg       0.98      0.94      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```


963
3
17
132
ham
spam
Actual Labels
ham
spam
Predicted Labels
800
600
400
200